

COMPETITIVE UX RESEARCH · FULL-APP QA
REPORT

Nut AI vs the Market

How Cal AI redefines effortless logging, what the best workout trackers do, and a tracked register of 25 issues — with a concrete reproduction roadmap. Evaluation only; no code changed.

Nut AI Engineering

Baseline v0.3.0 · main @ e354451 · SobhnathxLuffy/nut-ai

OCTOBER 2026 · EVALUATION ROUND

Table of Contents

1. Executive Summary3

1.1 Top findings at a glance3

1.2 What the owner asked for vs what this report delivers4

2. Scope, Method and Constraints5

2.1 Method and tooling5

2.2 Device-testing constraints, verified fresh5

2.3 Evidence conventions used throughout6

3. Cal AI Teardown: The Benchmark for Effortless Logging6

3.1 The core loop: snap, see, move on7

3.2 Design language and retention surfaces7

3.3 The seven principles we can copy8

3.4 Where Nut AI stands today, measured the same way8

4. Workout Tracker Landscape: What the Best Apps Do9

4.1 Logging UX: the three-tap law9

4.2 Progression intelligence: multiple axes, visible reasons10

4.3 Beginners vs advanced users10

4.4 RPE, RIR and the R-words: four market strategies10

4.5 What people love most, ranked11

5. Training and Schedule System: Evaluation Findings11

5.1 Verdicts on the owner complaints11

5.2 The progression engine is capable but silent12

5.3 Adding an exercise: a walkthrough of the friction13

5.4 Data-model critique13

5.5 Terminology audit: every jargon instance a user meets13

5.6 Friction register14

6. Food Logging, Scan and AI Editing: Evaluation Findings15

6.1 Verdicts on the owner complaints15

6.2 The current scan state machine, measured like Chapter 316

6.3 Why the AI describe-and-edit feels broken: twelve failure modes16

6.4 Friction register	17
6.5 Standards-axis notes worth carrying into the fix wave	18
7. Device QA Status and Static QA Findings	18
7.1 Environment and emulator attempt (fresh verification)	18
7.2 GitHub release state (facts, via API)	19
7.3 Static journey QA: five critical paths traced in code	19
7.4 Screen inventory sanity	20
8. Consolidated Issue Register	20
8.1 Register by area	21
8.2 Positive control	23
9. Reproduction Roadmap	23
9.1 From Cal AI: thirteen patterns for the food side	23
9.2 From the workout leaders: fifteen patterns for the training side	24
9.3 Suggested phasing	25
9.4 Anti-patterns to avoid while rebuilding	26
10. Prioritized Recommendations	26
10.1 Decision points for the owner	27

1. Executive Summary

This report answers the owner's latest directive for Nut AI: do not fix anything this round - evaluate everything. It combines a competitive research deep-dive into Cal AI and the leading workout tracking apps with a full, evidence-cited evaluation of our own training, scheduling, food-logging, scanning and AI-editing flows at baseline v0.3.0 (main @ e354451). Every finding is tied to a file and line number, a severity, and a concrete direction, so the next implementation wave can be planned directly from this document without re-doing the analysis.

25 issues tracked	5 P0 blocking	9 P1 major	11 P2 polish	28 evidence files
----------------------	------------------	---------------	-----------------	----------------------

The headline finding is that Nut AI's problems are concentrated in flow design, not in missing engine capability. The training side already ships a five-kind progression engine, plate-aware math, previous-workout prefill and undo/redo - but the engine runs silently at routine launch, skips every non-barbell exercise, and never explains itself. The scan side already carries an optimistic-log scaffolding (an analysis_status='captured' state with pending-count UI copy) that no production code path ever writes - the exact mechanism Cal AI's winning flow is built on. In both cases the capability exists and the user experience fails to expose it.

Competitive research shows a consistent playbook. Cal AI, now a 15M-download, roughly \$30M-ARR business acquired by MyFitnessPal, treats the AI's first answer as the log entry by default and makes correction a cheap, always-available overlay - log now, fix later. The best gym trackers (Hevy, Strong, Fitbod, Boostcamp, Alpha Progression) compete on a three-tap set loop, previous-workout values shown inline, multi-axis progression (weight, reps, sets, RPE), and either hiding RPE/RIR jargon behind plain language or teaching it at the point of use. Nut AI currently does none of these things end to end, and the owner's four training complaints and five food/score complaints were all confirmed against the code.

Mandate compliance

No repository file was modified in this round. Five sub-agents ran under a read-only constraint: two web-research analysts (Cal AI teardown, workout-app landscape), two code evaluators using the mattpocock two-axis method (spec axis vs the owner's complaints, standards axis for smells), and one device-QA engineer. All issues, including any found incidentally, were recorded rather than fixed.

1.1 Top findings at a glance

- Scan flow is the single biggest gap vs Cal AI.** Our camera result screen is a full-screen native modal that blocks the app for the entire 10-180 s analysis window, then demands an explicit confirm before anything is logged. Cal AI commits the entry and returns control to the user in seconds.

- **Quick/advanced mode is a confirmed dual-path subsystem** (persisted preference, viewfinder toggle, two divergent render trees of roughly 130 and 440 lines) that silently demotes scans and doubles the maintenance surface. The owner's instruction to delete it is technically straightforward.
- **Progression is real but invisible.** nextProgression() supports five kinds, yet fires only at routine launch, only against history from the same routine, and gates on load being non-null - so bodyweight, timed and assisted exercises never progress. The engine's explanation string is discarded.
- **Logged-meal macros are immutable.** updateLoggedMeal() writes only display_name and grams; there is no UI or API to correct calories or macros of an item after logging, and no AI command can either.
- **The AI describe-and-edit path has twelve distinct failure modes**, including a reply that renders "Done!" while silently discarding its own tool call, raw item ids shown to users, and a non-streaming 30-second correction request aimed at a gateway documented to kill non-streaming calls at ~30 s.
- **Device QA remains honestly NOT VERIFIED.** The sandbox has no /dev/kvm; the emulator now passes its compatibility checks under pure-software TCG emulation but dies silently mid-boot (4 attempts, logs preserved). Release state was verified via API: tag apk-v0.3.0 exists with a green CI build.

1.2 What the owner asked for vs what this report delivers

Owner request	Where it is answered
Evaluate the program/routine system, exercise adding, weight-only incrementation, RIR/RPE jargon	Chapter 5 (code verdicts with file:line), Chapter 9.2 (multi-axis fix)
Evaluate food-logging UX, the quick/advanced mode, the locked-behind-a-screen scan flow, and the weak describe-and-update feature	Chapter 6 (state machine + 12 failure modes), Chapter 9.1
Research Cal AI UI/UX and functionality and how to reproduce it here	Chapter 3 (teardown), Chapter 9.1 (13 patterns)
Research the best workout apps for beginners and advanced lifters, their UI/UX, and what people like	Chapter 4 (landscape + ranked patterns), Chapter 9.2
QA the actual Android app and track every issue	Chapter 7 (device attempt + static journeys), Chapter 8 (register of 25)
Produce a full PDF report and push it to the repo	This document, committed under docs/reports/ on main

Table 1-1: Traceability from the owner mandate to report chapters.

2. Scope, Method and Constraints

This chapter records how the evaluation was produced so its findings can be trusted, repeated, and audited. The round was explicitly evaluation-only: five sub-agents were dispatched in parallel, each with a hard no-write constraint on the repository, and each required to log its work to the shared worklog and return a full digest. The baseline is main @ e354451 - the v0.3.0 documentation commit that also carries the six owner-reported fixes from the previous wave (favorites-only shortcuts, logged-meal calories and macros on tap, the 116-SKU branded barcode corpus, multi-food AI logging, and the composer reachability and keyboard-avoidance fixes).

2.1 Method and tooling

- **Competitive research (T1-a, T1-b).** Two analysts ran a combined 40+ web searches through the web-search CLI, saving raw JSON evidence under research/raw-*.json, then wrote structured teardowns (cal-ai-teardown.md, 3,115 words; workout-apps-teardown.md, 3,020 words). Claims are cite-backed to source domain and date; the handful of inferences are flagged as such in the source files.
- **Code evaluation (T2-a, T2-b).** Two evaluators applied the mattleck two-axis method: the spec axis measures the implementation against the owner's stated complaints; the standards axis records Fowler-style smells as judgement calls. Both walked flows the way a user would, counting taps and screens, and produced friction registers ordered by severity.
- **Device QA (T3).** A time-boxed (~35 min) engineer re-verified the Android-environment constraints from scratch in the reset sandbox, attempted a software-rendered emulator boot, then pivoted to a static journey QA of five critical user paths plus a GitHub release-state audit.
- **Skills mandate.** ponytail (lazy-minimal discipline), mattleck-code-review (two-axis aggregation), web-search and the pdf Report pipeline were loaded and actually used; this document itself is the pdf skill's output, generated through palette.cascade, the HTML/Playwright cover system and ReportLab.

2.2 Device-testing constraints, verified fresh

The sandbox exposes no /dev/kvm and no vmx/svm CPU flags, so hardware-accelerated Android virtualization is impossible here. This round did produce one correction to the previous session's record: the modern emulator (37.2.12) does not hard-refuse x86_64 images without KVM - it accepts a -no-accel launch and enters genuine TCG software emulation. The practical wall remains: across four launcher runs the guest never reached its first Android frame, dying silently two to three minutes into boot with no fatal log line and an empty crashpad store. The 150 MB v0.3.0 APK was deliberately not downloaded since there was nothing to install it on and disk had to be preserved.

Check	Result	Consequence
/dev/kvm, vmx/svm flags	Absent (2-vCPU Xeon, nested virt not exposed)	No KVM path
Emulator 37.2.12 launcher checks	Pass without KVM; warns about 2 cores	TCG softemu attempted
TCG boot, 4 runs	Silent death 2-3 min in; no minidump	Device rows NOT VERIFIED
Disk / RAM	8.7 GB free / 4.0 GB RAM at start	APK download skipped
GitHub releases (API)	apk-v0.3.0 Release live, APK + sha256 uploaded; CI green on apk-v0.3.0 and main	Release pipeline healthy

Table 2-1: Fresh environment verification (T3).

2.3 Evidence conventions used throughout

Every code claim in Chapters 5-8 carries a file:line citation against e354451 so a reviewer can jump straight to the source. Severity is defined as follows: P0 means the flow blocks, traps, or silently corrupts data in a way that breaks trust or prevents the core job; P1 means major friction that will drive ordinary users away or make a headline feature effectively unusable; P2 means polish, consistency, or secondary-flow issues. Research claims in Chapters 3-4 cite their source domain and, where known, date; they are evidence-backed reconstructions of competitor behavior, not line-level certainties, and are labelled as estimates wherever a number could not be fully verified.

3. Cal AI Teardown: The Benchmark for Effortless Logging

Cal AI is the reference point the owner named, and the evidence says that reference is well chosen. Launched in May 2024 by two 17-year-old founders, it passed 5M downloads within about eight months, reached roughly 15M downloads and an estimated \$30M annual revenue run-rate by late 2025, and was acquired by MyFitnessPal with the deal announced in March 2026 - retained as a standalone app. Its promise is a single sentence: snap a photo, get the breakdown. Everything about its design serves that promise, and the instructive part for us is not the AI accuracy but the flow architecture around it.

15M+

downloads

~\$30M

ARR (2025)

\$2.49/mo

entry pricing (2025)

3-5 s

analysis wait

3.1 The core loop: snap, see, move on

The loop starts at the camera, not at a plus-menu. The user snaps; a designed analysis screen covers the 3-5 second model round-trip (Cal AI's own blog bounds the worst case under 15 seconds); the result appears as a card listing every food detected in the scan with per-item quantity chips; and one tap commits it. The decisive design choice, triangulated across the App Store how-to, third-party teardowns and QA guides built on top of Cal AI, is that there is no separate review gate: the save is the default action, and the sanctioned working model expressed by testing guides is log now, fix later.

Step	What happens	User cost
1. Snap	Camera is the app's front door; shutter is the hero control	1 tap
2. Wait	Staged analyzing screen (detecting, estimating, calculating) covers the 3-5 s call	0 taps, bounded
3. Result	Card lists all detected foods with macros; quantities editable in place	0-2 taps
4. Commit	One primary save writes the entry; user is back on the dashboard	1 tap
5. Correct	Any time later: tap the meal card to edit items, or tell the AI chat what to fix	1-2 taps, async

Table 3-1: Cal AI scan-to-logged loop. The wait is designed; correction is deferred, not gated.

Editing later is a first-class path, not an apology. Tapping a logged meal opens an Edit Items list where quantities can be adjusted, items renamed or removed; and the same conversational input that logs food by description ("describe your meal") doubles as a correction channel. Fallbacks are equal verbs rather than hidden modes: photo, barcode, text description and voice all lead into the same log. The strategic asymmetry is deliberate - the 50-times-per-day action (camera) is optimized to seconds, while the once-per-day deep-search path is measurably slower (about 19 actions in one teardown's measurement) and nobody minds.

3.2 Design language and retention surfaces

The dashboard is a glanceable identity: a large remaining-calorie ring, macro bands, and meal cards, rendered dark, minimal and high-contrast in a single accent - a screenshot-friendly aesthetic that doubles as the brand's TikTok ad creative. Onboarding is a roughly 29-step quiz that manufactures commitment and computes macro targets before the paywall; streaks and ring completion (rather than badge grids) provide the habit loop; and home-screen ring widgets keep the identity visible without opening the app. Users praise exactly this: speed, effortlessness, no manual typing, and the design itself. They complain about mixed-dish accuracy (hidden oil, off-frame ingredients), occasional ignoring of user-provided grams, subscription resentment at the ~\$9.99/mo AI wall, and weakness as a general workout tracker - which is precisely the opening for a nutrition-plus-training app like ours.

3.3 The seven principles we can copy

- 1 **Default-to-logged, not default-to-review.** The AI's estimate is the entry; confirmation is a cheap afterthought, never a gate.
- 2 **Friction asymmetry.** Optimize the 50x/day action ruthlessly; accept slowness in the 1x/day action.
- 3 **Design the wait.** A staged status screen makes a 3-5 s model call feel intentional, not broken.
- 4 **Correction is a habit, not a checkpoint.** Fix misses any time before the day closes.
- 5 **Glanceable identity.** Ring + streak + widgets retain users independently of any single log.
- 6 **Onboarding as investment.** A commitment device that also computes the plan the app will use.
- 7 **Badge estimates honestly.** Mark AI outputs as estimates and make correcting them effortless - accuracy complaints are survivable when editing is cheap; silent wrongness is not.

3.4 Where Nut AI stands today, measured the same way

Applying the same measurement to our current scan flow gives: best case three taps, but the entire analysis window is a blocking full-screen modal with no route back to the app, and nothing is written until the user confirms on the review screen. Manual search logging costs five-plus interactions plus typing. The chart below makes the structural gap visible: our scan path matches Cal AI's tap count on paper while being categorically heavier in experienced effort, because every second of our wait holds the user hostage and every log demands a decision. Chapter 6 dissects the mechanism; Chapter 9.1 gives the thirteen-pattern repair plan.

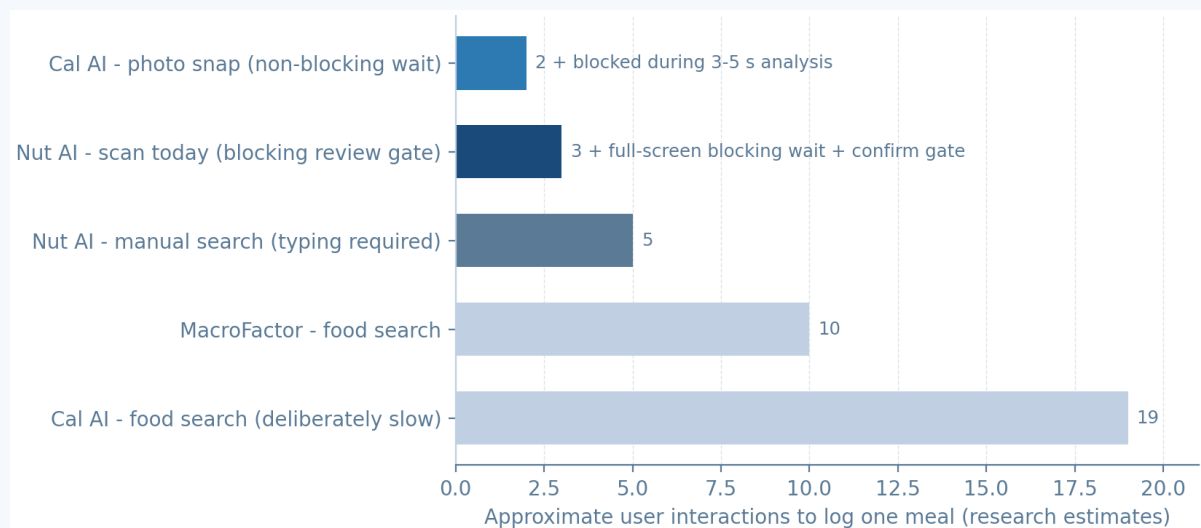


Figure 3-1: Approximate interactions to log one meal (research estimates; annotation shows blocking behavior). Cal AI search being slow is deliberate - the camera path carries the load.

4. Workout Tracker Landscape: What the Best Apps Do

The workout-tracker market splits into three tiers, and each tier teaches a different lesson. Pure loggers (Strong, Hevy, StrengthLog) compete on the fastest possible set entry. AI planners (Fitbod, FitnessAI, Alpha Progression, Dr. Muscle) decide what you should lift next. Program libraries (Boostcamp, StrongLifts) hand you a proven plan and automate its arithmetic. Nut AI's training side currently sits awkwardly between tiers: it has planner machinery (the progression engine) but logger-grade UI friction (a fifteen-control mega-form for editing routines), which is exactly the mismatch the owner flagged.

App	Audience	Model	Standout
Hevy	Beginner to advanced	Freemium; Pro ~\$24/yr	Previous-workout values inline while logging; 17M users; the most-copied UX in the category
Strong	All levels	PRO \$4.99/mo	Three-tap set loop; plate + warm-up calculators as the paid layer; benchmark logging speed
Fitbod	Beginner/intermediate	Subscription	Multi-axis AI progression (weight, reps, sets) plus a muscle-recovery heatmap
JEFIT	All levels	Freemium	~1,400-exercise library with demos; calendar-view history
Boostcamp	Beginner to advanced	Free	11,000+ programs incl. 130+ coach-designed, with inline coaching notes; GGR's pick for most users
Alpha Progression	Intermediate to advanced	Freemium	Explicit double progression: weight AND rep recommendations per exercise
StrongLifts 5x5	Absolute beginners	~\$30/yr	Radical simplicity: fixed rules, auto 2.5 kg increments, automatic deload; never mentions RPE
StrengthLog	Data-minded	Generous free tier	Unlimited logging + PRs; volume-by-muscle monitoring

Table 4-1: Market overview (evidence: loadmuscle, repretun, Garage Gym Reviews, setgraph, App Store/Play synthesis).

4.1 Logging UX: the three-tap law

Every leader converges on the same logging physics. Sets are entered with big plus/minus steppers and a large check button; the previous workout's values sit ghosted in each input so the user usually taps check three times; duplicating the last set is one gesture; set types (warm-up, drop, failure) are first-class rather than buried; and an auto rest timer starts the moment a set is checked. Strong monetizes the moments around the set - plate calculator, warm-up calculator, insights - precisely because the logging itself must stay free and frictionless. Hevy adds smart superset scrolling (checking a set auto-scrolls to the next superset exercise) and shows one-rep-max percentages so the user can decide progression with context. The shared lesson: in-session typing is the enemy; every field defaults to something sensible and every change is a stepper away.

4.2 Progression intelligence: multiple axes, visible reasons

The planners differentiate on progression, and the shape of that differentiation maps one-to-one onto our owner's complaint. Fitbod moves weight, reps and sets together, computed from volume, frequency, intensity and recovery - powerful but opaque. Alpha Progression champions double progression and says so out loud: add reps within a range, then add load - with a visible next-time recommendation the user can override. StrongLifts picks a single rule (2.5 kg per session) and enforces deloads automatically. Gymverse makes auto-increase of reps and weight a headline marketing claim for mainstream users. What nobody successful does is what we do: compute a progression silently, apply it only to barbell lifts, only at routine launch, only against the same routine's history, and throw the explanation away. The market either decides for you visibly or gives you the numbers to decide; it never decides for you invisibly.

4.3 Beginners vs advanced users

- **Beginners want decisions made for them:** what weight to start at, when to increase, what to do when they stall. They want demo videos, plain language, and short feedback loops (streaks, simple charts). StrongLifts and Boostcamp win here by replacing scales with rules and coach notes.
- **Advanced lifters want control and data:** custom routines, supersets, per-set rep targets, RPE/RIR fields, 1RM percentages, plate math, warm-up ramps, rest-timer flexibility, volume by muscle, export. They log the most sets, which is why Strong's three-tap loop keeps them.
- **The gap in the middle is our opportunity.** The winning posture - Hevy's, and partially Alpha's - is to default simple (auto progression, plain language) and unlock numeric RPE targets, custom routines and granular progression when the user is ready. Nut AI's nutrition-plus-training combination is a natural home for that posture; no surveyed leader owns both sides.

4.4 RPE, RIR and the R-words: four market strategies

RPE is a 1-10 rating of how hard a set felt; RIR is how many clean reps were left in the tank (RPE 8 is roughly RIR 2). The market handles these terms in four distinct ways, and only one of them is wrong - ours. Strategy one is to hide the scales entirely behind rules (StrongLifts) or coach notes (Boostcamp: when the last rep feels hard, add 2.5 kg). Strategy two is an optional numeric field with a plain-language glossary at the point of use (Hevy). Strategy three is to compute effort for the user and output concrete prescriptions like 3x8 @ 50 kg (Fitbod, Alpha, JuggernautAI). Strategy four - the one we should steal - is a post-set subjective prompt, How hard was that? Easy / Could do 2 more / Barely finished, mapped internally to RPE ~6/8/9.5, with an advanced toggle exposing numeric RPE/RIR and a paired converter. The anti-pattern is shipping required, unexplained abbreviation fields - which, per Chapter 5's terminology audit, is what Nut AI does today across routine fields, live set rows and error messages.

4.5 What people love most, ranked

#	Pattern	Champion app
1	Previous-workout values shown inline while logging a live session	Hevy
2	One-tap set check + duplicate last set (three-tap loop)	Strong / Hevy
3	What-to-lift-next auto recommendation with visible reasons	Fitbod / Alpha / Gymverse
4	Free, unlimited-enough core logging	Hevy / Boostcamp / StrengthLog
5	Coach-authored programs with day-by-day inline guidance	Boostcamp / StrongLifts
6	Plate calculator with plates-per-side visual; warm-up calculators	Strong PRO / Hevy
7	Auto rest timer that starts on set check	Hevy / Strong / StrengthLog
8	Muscle recovery / volume heatmap	Fitbod / StrengthLog
9	Social feed, PR celebration, year-in-review	Hevy / JEFIT
10	Smart superset scrolling; first-class set types	Hevy / Strong
11	Calendar history view	JEFIT
12	Watch and home-screen widgets surfacing last workout / start workout	SmartGym (Apple ads)

Table 4-2: Cross-app pattern ranking from review synthesis and teardowns.

5. Training and Schedule System: Evaluation Findings

This chapter evaluates the Programs / Routines / Workout subsystem against the owner's four complaints using the two-axis method. The overall verdict: all four complaints are confirmed against the code, with one technical nuance - a reps-then-load (double progression) kind does exist in the engine - but the nuance is invisible at the point of use, so the owner's description matches the shipped experience. The fairness note matters too: the live workout screen is a competent Hevy-style set table with previous values, auto-advance, a rest chip and workout-scoped undo/redo; the failures concentrate in everything around it.

5.1 Verdicts on the owner complaints

Complaint	Verdict	Root cause (file:line)
Program system 'not suitable at all, like the adding of exercises'	CONFIRM ED	Routine editor is one flat mega-form with ~15+ inline controls per exercise (routines.tsx:602-763); adding exercises leaves the editor for the generic search screen (routines.tsx:767-780 to search.tsx:246-286) and returns results through a module global (src/data/routine-draft.ts, merged at routines.tsx:337-368); every added exercise arrives as three hard-coded 20 kg x 10 reps sets (routines.tsx:305-314, duplicated in packages/training/src/repository.ts:161-170 and log-exercise.tsx:312-324); no reorder inside the editor (Remove only, routines.tsx:605).
Only one incrementation way - weight, no rep way	CONFIRM ED (nuance)	nextProgression() gates on load !== null (packages/training/src/progression.ts:39) so bodyweight, reps, time and assisted exercises never progress. A 'double' kind does add reps until max_reps then bumps load (progression.ts:42) but fires only at routine launch (repository.ts:358), only against history of the same routine_id (repository.ts:352), and discards the engine's explanation string. Routines saved from a workout are hard-coded kind:'manual' (workout.tsx:339) so never progress.
People don't understand RIR, RPE and the different R's	CONFIRM ED	RIR/RPE appear unexplained in routine fields and errors (routines.tsx:38,445,479,683,703-709), live set rows (workout.tsx:780-791), and a toggle caption (workout.tsx:386-387); raw set-kind enums render verbatim: warmup/drop/failure/amrap/myo/clusterr/cooldown (workout.tsx:767-775, core-schema/src/training.ts:5); the Epley formula name shows in reports (ReportScreen.tsx:140). Definitions exist only in an internal doc; the tutorial card is three generic sentences.
Routines/schedules/programs UI-UX not good while adding exercises	CONFIRM ED	Program creation is a form of three numeric date fields plus weeks plus a 7-row weekday x routine chip matrix (programs.tsx:197-254) with no date picker and no calendar preview. Five overlapping workout-launch paths exist (routines.tsx:826, train.tsx:21, train.tsx:25, programs.tsx:318, plus start-empty at train.tsx:18). Deleting a routine soft-deletes only its row (routines.tsx:536-544) while the dialog promises removal from programs (routines.tsx:840) - the dangling reference then throws 'Routine not found' at launch (repository.ts:346).

Table 5-1: Spec-axis verdicts with primary evidence.

5.2 The progression engine is capable but silent

packages/training/src/progression.ts implements five kinds (fixed, linear, double, percentage, RIR-based) with plate-aware rounding and per-set-slot prefill - genuinely more machinery than the complaints imply. Three design choices make it feel absent. First, timing: it runs only inside routine launch (repository.ts:358), so there is no suggestion during a live set and no next-time banner after an exercise. Second, scope: history is queried with WHERE routine_id = ? (repository.ts:352), so the same movement logged under a different routine contributes nothing. Third, silence: the engine computes an explanation string that is discarded at the call site, and planned_json is overwritten with the progressed values (repository.ts:360), destroying the planned-vs-actual comparison that the internal acceptance criteria (TRN-005) call for. The market pattern from Chapter 4.2 is the inverse: decide visibly, show the reason, let the user override.

5.3 Adding an exercise: a walkthrough of the friction

- 1 Open the routine editor - a single scrolling form, not a list-plus-detail.
- 2 Tap '+ Select exercise'; the app leaves the editor and pushes the generic search screen.
- 3 Search and tap a result; the handoff flows through the routine-draft module global, not a parameter.
- 4 Back in the editor, the new exercise appears with three hard-coded 20 kg x 10 reps sets, regardless of the user's history with that lift or their unit setting (the editor is unit-blind: getFieldLabels('kg') hard-coded at routines.tsx:600 while the live workout is unit-aware at workout.tsx:118,124).
- 5 To reorder, there is no control in the editor at all - reordering exists only in the live-workout menu (workout.tsx:233-245); the editor offers Remove (routines.tsx:605).
- 6 Repeat for each exercise. A one-exercise routine costs roughly ten interactions plus two context switches.

The unit-blindness is a silent data-corruption bug, not just UX

Because the routine editor hard-codes kg labels with no conversion while the live session converts, a lb user who plans 100 (lb) will find the session interpreting it as 100 kg - a 2.2x planning error that surfaces mid-workout.

Severity: P0-adjacent; it silently corrupts intent for every non-kg user.

5.4 Data-model critique

The subsystem has eight training tables, but routines and programs are two structurally identical one-row JSON-blob tables - literally export type Program = Routine (repository.ts:12) - re-parsed at render time in at least five sites with three inconsistent failure behaviors (silent no-op on a corrupt row at routines.tsx:277-279, an unreadable card at programs.tsx:282-292, and an untyped parse at exercise-detail.tsx:200). Schedule days are stored as cycle days requiring weekday-to-cycle conversion at every boundary (repository.ts:319-324), a second day-language born from an earlier owner bug. Three conflicting notions of 'previous' coexist: workoutDetail.previous queries any prior workout (repository.ts:212), previousBySetSlot uses set-slot identity (set-table.ts:33), and launch history uses same-routine only (repository.ts:352). A simpler shape - relational routine_exercises/routine_sets rows; Program(start_date, weeks) plus program_days(weekday, routine_id) as foreign-key-protected real rows in one day language; and a single 'prepare next session' function that stores the plan, the progression decision and its reason on the session - would eliminate most of the failure surface catalogued above.

5.5 Terminology audit: every jargon instance a user meets

Term as shipped	Where it hits the user
'RIR' chip; 'Target RIR (0-10)'; 'Planned RIR/RPE'	Routine editor fields and live set rows
'RIR runs 0-10' / 'RPE runs 1-10' errors	Validation messages - jargon taught by scolding
Set kinds rendered raw: amrap, myo, cluster, warmup, drop, failure, cooldown	Live workout set-type picker
Progression chips 'Double / Fixed / Percentage / RIR / Manual'	Routine editor - engine taxonomy, not user outcomes
'Estimated 1RM (Epley)'	Progress reports - formula names instead of meaning
Tempo '3-1-2-0' with no decoding	Routine editor fields

Table 5-2: All unexplained in-app; definitions live only in an internal glossary document.

5.6 Friction register

ID	Sev	Issue	Evidence
T1	P0	Progression is engine-only and invisible: silent launch-time prefill, skipped for all non-loaded lifts and outside the same routine, explanation discarded	progression.ts:39; repository.ts:352,358
T2	P0	Adding exercises = leave-editor context switch + hard 20 kg defaults + no reorder; ~10 interactions minimum	routines.tsx:305-314,605,767-780
T3	P0	Routine editor unit-blind (kg hard-coded) while live session is unit-aware - 2.2x silent planning error for lb users	routines.tsx:600; workout.tsx:118, 124
T4	P1	Routine delete leaves a dangling program schedule; error only at launch tap	routines.tsx:536-544,840; repository.ts:346
T5	P1	RIR/RPE/set-kind/1RM jargon with zero in-app education	Table 5-2 references
T6	P1	Program creation is a form, not a plan; no 'Week N of M' exists anywhere - weeks only gates the finished state	programs.tsx:197-254; repository.ts:338
T7	P1	'Save as routine' always mints a NEW manual-rule routine - near-duplicates accumulate and can never progress	workout.tsx:327-350
T8	P2	planned_json overwritten with progressed values at launch; planned-vs-actual destroyed	repository.ts:360
T9	P2	Two parallel exercise universes (strength journal vs MET-calorie entries); MET configure defaults 20 kg/10 reps despite rich history	log-exercise.tsx:79-84,312-324
T10	P2	Vocabulary drift: sessions still default-named 'Quick workout' vs the retitled 'Start empty workout'	repository.ts:198
T11	P2	Editor affordances missing: no reorder, duplicate, per-exercise rest, or collapse	routines.tsx:602-763

ID	Sev	Issue	Evidence
T12	P2	Progress side does not interlock: no streaks feature exists; Training progress is aggregate-only with no program adherence or scheduled-vs-done calendar	progress.tsx:171-178

Table 5-3: Training friction register (T1-T12).

6. Food Logging, Scan and AI Editing: Evaluation Findings

This chapter evaluates the food side against the owner's five complaints. All five are confirmed, and the code contains a striking discovery: the Cal AI-style optimistic flow is already scaffolded but never wired. The database defaults new meals to `analysis_status='captured'` (db-adapter/schema.ts:221), the day-total pipeline counts captured/queued/analyzing rows as a `pendingCount` contributing zero calories (repo.ts:305-317), and day-detail already renders copy for items 'still being analysed' (day-detail.tsx:163-167). No production write path ever inserts a pending meal - the scaffold reads but nothing writes. The fix direction is therefore not new machinery but connecting existing pieces in a different order.

6.1 Verdicts on the owner complaints

Complaint	Verdict	Root cause (file:line)
Food logging has UI/UX issues	CONFIRMED	Ten-item friction register below (F1-F10), each with file:line evidence.
There should not be any quick and advanced mode	CONFIRMED	Quick/advanced is a first-class subsystem: persisted preference (camera.tsx:28-52), viewfinder toggle (camera.tsx:130-167, rendered at : 277), per-scan handoff (camera.tsx:240,261 to src/scan/store.ts:421-438), consumed via viewOverride ?? initialReviewMode (result.tsx:227-228,350), with two divergent render trees (result.tsx:454-581 quick vs 583-1023 advanced) and a fallback notice (:606-612). Eligibility (quick-mode.ts:24-31) silently demotes any scan with an AI-estimate row or band outside 15%.
Should be like Cal AI: snap, it shows in the log, keep using the app, no lock-in	CONFIRMED	Food reaches the DB only after the model answers AND the user taps 'Log it': shutter routes to /result via router.replace (camera.tsx:245); /result is a full-screen native modal (_layout.tsx:123-124); analyzing occupies the whole screen (result.tsx:254-286); the only write is logMeal() (result.tsx:385-391, repo.ts:393-401). Backing out abandons the flow with no completion notification; gateway scans can run to GATEWAY_SCAN_TIMEOUT_MS = 180,000 ms (timeouts.ts:46).
Edit macros or weight of a logged item	PARTIAL	Weight: yes (meal-detail.tsx:423-428 via updateLoggedMeal, logged-meals.ts:72-87). Macros: impossible - updateLoggedMeal writes only display_name and grams (logged-meals.ts:78-81); the editor has exactly two fields; even scan review edits only grams/band (result.tsx:813-830,1568-1603).

Complaint	Verdict	Root cause (file:line)
The describing-and-updating feature does not work well	CONFIRMED	Twelve failure modes across two paths, audited in 6.4 below.

Table 6-1: Spec-axis verdicts with primary evidence.

6.2 The current scan state machine, measured like Chapter 3

- 1 /camera (modal): four mode pills (food/barcode/label/receipt) plus two review-mode pills; 1 required tap plus up to 4 optional ones stacked over the shutter (camera.tsx:276-304).
- 2 capture(): haptic, in-memory phase (not a DB row, despite the camera.tsx:174-177 comment claiming a 'draft row' is written), then router.replace('/result') and an async startScan().
- 3 /result analyzing: full-screen blocking; model window 10 s+ with gateway scans up to 180 s; tabs are under a modal; 'Close' abandons the scan with no route back and no notification (an epoch guard then supersedes it if the user retries).
- 4 Eligibility branch: isQuickEligible() requires at least one ingredient, zero highlighted questions, zero AI-estimate rows, band within 15%; any miss silently demotes to the advanced tree.
- 5 Review: grams inputs, remove rows, question chips, add-ingredient sheet, fix-result sheet, band quick-sets, plus a background web-lookup refinement that can change numbers mid-review.
- 6 'Log it': logMeal() transaction, slot derived from the clock, dismissAll, a 6-second undo toast. Failure states offer honest Retry/Retake/Log-manually escapes.

The structural comparison with Cal AI is therefore not about tap counts - our best case is three - but about ownership of time and decisions. Cal AI writes the entry and hands control back before the user can get bored; we hold the user in a modal for the entire analysis and demand a confirmation decision before anything exists. Chapter 9.1's first three patterns (optimistic insert at shutter time, dismiss-on-capture, review as an editor over the logged row) reverse exactly this, and the dead 'captured' scaffold means the database already knows how to represent the in-flight state.

6.3 Why the AI describe-and-edit feels broken: twelve failure modes

#	Failure mode	Evidence
1	Today-only world: the assistant ships only today's items in context and re-resolves keys against today only - past-day edits are structurally impossible	assistant.ts:228-274; assistant.tsx:542; log-corrections.ts:100-106
2	A reply containing both text and a tool call renders the text and discards the call - the user sees 'Done!' while nothing was applied	assistant.ts:47-53
3	Silent partial application: unknown ids and missing-gram ops go to an unknown[] that is never surfaced; the card flips to 'Applied' while skipped ops vanish	assistant.tsx:553-578

#	Failure mode	Evidence
4	Raw ids in the confirm card: nameOf = (id) => id renders 'Set m12i3 to 200 g' - unverifiable before confirming	assistant.tsx:857,906
5	qualitative_size is dead schema: ops with grams == null are dropped, so 'make it two rotis' can never work	core-schema/correction.ts:5-9; assistant.tsx:556-560
6	Ephemeral proposals: tool cards and status are memory-only; history replays text only	assistant.tsx:218-227
7	Fragile JSON extraction: first-brace-to-last-brace parsing; the streaming guard only knows one shape	assistant.ts:114-121,84-95
8	The cheapest configured model is trusted with an exact-id emit-and-echo contract	assistant.tsx:348-351
9	Fix-Result path costs 4+ interactions (type, Update, model, confirm, Apply); ambiguity loops force retyping	result.tsx:1041-1152
10	Weakest transport in the app: prose-only schema with no json_schema or repair, non-streaming with a 30 s timeout - aimed at the gateway documented to kill non-streaming calls at ~30 s; single attempt, no fallback	prompt/correction.ts:10-27; transports/openai.ts:127-146; calls/correction.ts:24; timeouts.ts:23
11	Id-echo contract: mismatch produces 'no longer in the list'	result.tsx:1112-1115
12	Skips collapse into one muted notice line with no per-op record	result.tsx:1148-1151

Table 6-2: AI-edit failure modes. Items 1-8: assistant chat path; 9-12: Fix-Result path.

Related surprise: "add a coffee" joins dinner

The assistant's add_item operation appends to the chronologically last meal of the day (assistant.tsx:544,563-567), so a morning coffee asked at 9 am silently lands in yesterday-evening's dinner slot if that was the last meal row. Slot inference should consider the mention's time words, not just row order. Severity: P1 trust issue.

6.4 Friction register

ID	Sev	Issue	Evidence
F1	P0	Scan locks the app behind a full-screen modal; the log lands only after the whole wait plus an explicit confirm	camera.tsx:245; _layout.tsx:123-124; result.tsx:254-286,385-432
F2	P0	Quick/advanced dual-path subsystem to delete; silent demotions and duplicated render trees	camera.tsx:28-52,130-167; quick-mode.ts:24-31; result.tsx:454-1023
F3	P1	Logged-item macros immutable; editor exposes only name and grams	logged-meals.ts:78-81; meal-detail.tsx:422-428
F4	P1	Chat edits unreliable (twelve failure modes, Table 6-2)	see Table 6-2
F5	P1	Fix-Result correction call: non-streaming 30 s timeout on a gateway that kills non-streaming at ~30 s; no schema, no repair, no fallback	calls/correction.ts:24; timeouts.ts:23
F6	P1	AI add_item appends to the last meal of the day regardless of intended time	assistant.tsx:544,563-567

ID	Sev	Issue	Evidence
F7	P2	Meal slot clock-derived; meal-detail date is a free-text YYYY-MM-DD field with no date picker	date-utils.ts:22-27; meal-detail.tsx:277
F8	P2	Camera stacks six controls over the shutter	camera.tsx:276-304
F9	P2	Manual log path costs 5+ steps; one-tap strip covers repeats only	food.tsx:166-172,191-225; food-search.tsx:436-437; food-review.tsx:184-263
F10	P2	Undo lives only in a 6-second toast; afterwards correction requires the meal-detail journey	result.tsx:401-415; food.tsx:97-111

Table 6-3: Food/scan friction register (F1-F10).

6.5 Standards-axis notes worth carrying into the fix wave

Beyond the spec axis, the evaluator logged duplication that will multiply if the flows are patched piecemeal: the corpus-resolve chain appears three times (result.tsx:1223-1243, assistant.tsx:152-173, result.tsx:1201-1213); the undo-toast block with its identical 'Could not undo' string is copy-pasted three times; the quick and advanced trees duplicate hero, legend and macro render blocks; orchestrator.ts has grown to 1,093 lines with ~80 lines of label/receipt boilerplate duplicated internally; and there are two sources of truth for review mode (a settings row plus a module variable). Deleting the quick/advanced subsystem (F2) removes the largest chunk of this at once, which is a second reason to do it first.

7. Device QA Status and Static QA Findings

The owner asked for QA of the actual Android app. This chapter reports honestly what could and could not be verified on real hardware in this sandbox, then substitutes the strongest possible static alternative: a paper-device walk of every route plus code-traced simulations of five critical user journeys. The release state itself was verified live via the GitHub API and is healthy.

7.1 Environment and emulator attempt (fresh verification)

The environment re-check confirmed no /dev/kvm and no vmx/svm flags on the 2-vCPU Xeon host, 8.7 GB free disk and 4 GB RAM at start, with OpenJDK 21 installed. A leftover SDK at /tmp/my-project/android-sdk (emulator 37.2.12, cmdline-tools 12, a full android-34 google_api x86_64 image) was reused read-only. The notable correction to the prior session's record: the emulator launcher accepted a -no-accel x86_64 AVD and entered genuine TCG software emulation - the hard refusal applies only to arm64 images on this host. Across four runs (background and foreground, with and without -verbose, one with -feature -Vulkan,-GLDirectMem), the guest never reached the first Android frame: runs died silently at 88 s to ~3 min, the verbose log ends after SwiftShader/EGL initialization with zero fatal lines, and the crashpad database contains no minidump. Per the time-box the path was stopped; all logs are preserved

under qa/.

7.2 GitHub release state (facts, via API)

- Tag apk-v0.3.0 exists as a full (non-prerelease, non-draft) Release created 2026-10-06, with nut-ai-v0.3.0.apk (150,162,425 bytes, state 'uploaded') plus its .sha256 asset.
- CI: Build Android APK at apk-v0.3.0 (e354451) - success; web ci at main - success; rc1/rc2 builds also success. No red runs in the recent history.
- Nine tags total from v0.1.0 through v0.3.0 plus the rc chain. Nothing was created or modified during this evaluation round.

7.3 Static journey QA: five critical paths traced in code

Journey	What the code shows	Findings
J1 First launch to home	Gate at _layout.tsx:32-52 on kv key onboarding.completed.v1; draft persisted per step; plan finish persists targets before navigating and pushes the tutorial once.	F-QA1 (P1 candidate, device-verify): Back at step 1 exits to the tabs WITHOUT writing the done key - a 'skipped' user is re-onboarded on every cold launch and Home may render without persisted targets.
J2 Log food via search	food.tsx:171 to food-search.tsx:404-454 (low-confidence Indian-dish KB diverts to /dish-composer) to food-review.tsx:184-266: authoritative per-row grams, damaged saved-meal data blocked with honest copy, success toast carries Undo.	F-QA2 (P2): after saving, food-search remounts with prior results - the same food can be re-selected and re-logged; mitigation is undo, not prevention.
J3 Barcode offline hit/miss	Permission gate with explicit request and visible exit (camera.tsx:192-221); first barcode fires once; orchestrator.ts:643+ distinguishes offline vs genuine miss via a direct OpenFoodFacts probe; P2-37 3-step route contract-tested; honest Retry/Log-manually escapes.	F-QA3 (P2): camera.tsx:262-263 navigates to /result BEFORE startBarcodeScan, relying on the module-global epoch store - a mount/setPhase race window only a device test can clear.
J4 Routine to workout to finish	Train tab Start/Resume; routine Start disabled while a workout is active (no double-live-workout); corrupt program JSON degrades to a readable card; every routines exit runs a discard confirm; workout has planned-set pre-creation, prior-set cloning, undo/redo, discard confirm.	F-QA4 (positive): finishWorkout throws 'Complete at least one set' (repository.ts:244-248) - no empty completion, no double-finish. F-QA5 (P2, cosmetic): default name still 'Quick workout' (= T10).

Journey	What the code shows	Findings
J5 AI chat multi-food log and edit	Without a configured provider the assistant answers with an inline pointer to AI Provider settings (assistant.tsx:332-350) - an honest gate. Proposals route through resolveMealProposal (unit-count expansion, zero-ingredient guard) into food-review's explicit confirm; corrections are user-confirmed undoable ops; only PROPOSED/FAILED cards are applicable - no double-apply.	F-QA6 (P2): the journey hard-depends on a reachable LLM endpoint; offline the assistant cannot log or edit at all. The edit half additionally inherits every Table 6-2 failure mode.

Table 7-1: Journey-by-journey static QA. No P0 found statically; findings F-QA1..F-QA6 folded into the Chapter 8 register.

7.4 Screen inventory sanity

The route inventory is clean: five tabs (Home, Food, Train, Progress, You), a global FAB sheet with five actions, 36 stack routes, three deep-link aliases routing through DeepLinkRedirect, a +not-found catch-all with 'Go home', and a root ErrorBoundary with Try again / Go home. No trapped screens were found statically. What remains genuinely NOT VERIFIED without hardware: APK install and launch, real onboarding flow and tutorial overlays, tab rendering and haptics, the camera permission dialog, live barcode scanning against physical products, real OpenFoodFacts latency behavior, keyboard set entry, notifications and the rest timer, the home-screen widgets, live LLM streaming, and end-to-end performance. These rows stay explicitly marked NOT VERIFIED rather than assumed.

8. Consolidated Issue Register

This chapter merges every finding from Chapters 5-7 into a single tracked register: 25 issues - 5 P0, 9 P1, 11 P2 - plus one positive control worth keeping. IDs are stable and area-prefixed (T = training, F = food/scan, A = AI editing shares the F-IDs of its items in Chapter 6, QA = onboarding/cross-app) so they can be referenced in commit messages, issues, and the next wave's acceptance criteria. Severity follows the Chapter 2 definitions. Nothing in this register has been fixed; the register is the round's deliverable.

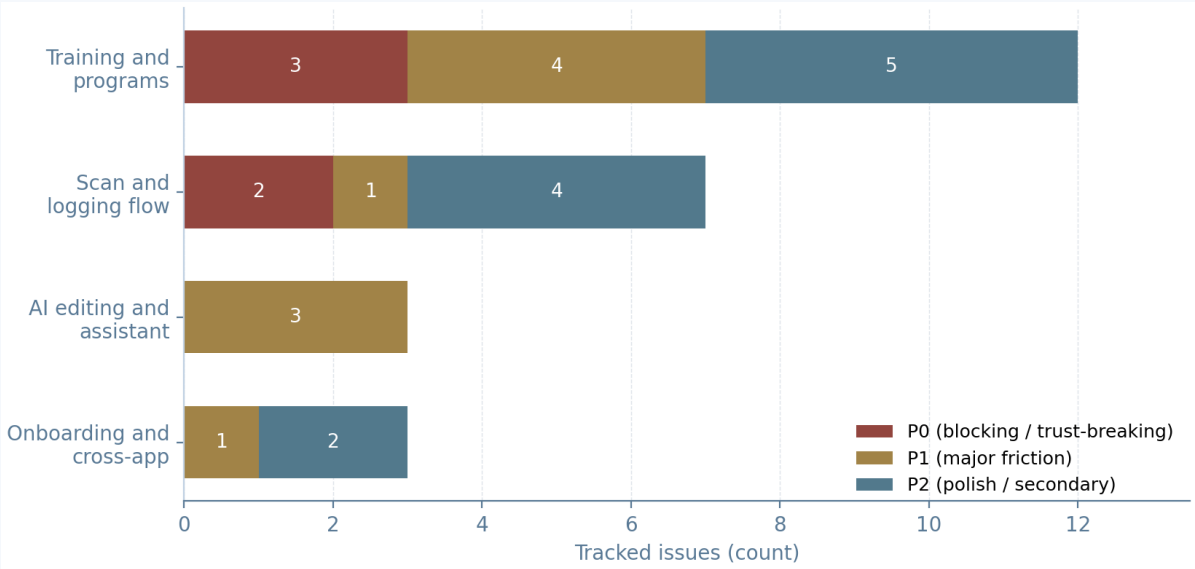


Figure 8-1: Tracked issues by area and severity. Training carries the most issues; scan/flow carries the two most user-visible P0s alongside training's three structural P0s.

8.1 Register by area

ID	Sev	Issue (short)	Direction
T1	P0	Progression silent, launch-only, same-routine-only, non-loaded lifts skipped	Visible suggestion chips with reasons; accept/edit/override
T2	P0	Exercise adding: context switch, hard 20 kg defaults, no reorder, ~10 interactions	In-editor picker sheet; history-aware defaults; drag reorder
T3	P0	Routine editor unit-blind vs unit-aware live session (2.2x silent error)	Unit-aware fields with conversion in the editor
T4	P1	Routine delete corrupts programs (dangling schedule)	Cascade cleanup or block with honest message
T5	P1	RIR/RPE/set-kind/IRM jargon unexplained everywhere	Plain-language capture + point-of-use explainers (Ch. 4.4)
T6	P1	Program creation is a form; no 'Week N of M' anywhere	Calendar preview + week indicator from start_date + weeks
T7	P1	Save-as-routine mints manual duplicates that never progress	Save over source routine or offer rule selection
T8	P2	planned_json overwritten at launch; planned-vs-actual lost	Store plan and progressed plan separately
T9	P2	Two parallel exercise universes (strength vs MET)	Unify entry models or bridge them
T10	P2	Default session name 'Quick workout' vs 'Start empty workout' UI	Rename constant; align vocabulary
T11	P2	Editor lacks reorder/duplicate/per-exercise rest/collapse	Add affordances when the model is normalized (Ch. 9.2)

ID	Sev	Issue (short)	Direction
T12	P2	Progress does not interlock with programs; no streaks/adherence	Adherence calendar + program adherence once model fixed

Table 8-1: Training and programs (12 issues).

ID	Sev	Issue (short)	Direction
F1	P0	Scan locks app in full-screen modal; log lands only after wait + confirm	Optimistic insert at shutter; dismiss-on-capture; pending card in timeline
F2	P0	Quick/advanced dual-path subsystem; silent demotions; duplicated trees	Delete the toggle; one automatic path with inline follow-ups
F3	P1	Logged-item macros immutable (grams/name only)	Per-100g macro overrides in meal-detail; totals already recompute
F4	P1	Chat edits unreliable - twelve failure modes (Table 6-2)	Fixes enumerated in Ch. 9.1 pattern 7
F5	P1	Fix-Result: non-streaming 30 s call on a gateway killing at ~30 s	Stream the call; add schema/repair; fallback chain
F6	P1	AI add_item appends to last meal of day regardless of time words	Time-aware slot resolution for add operations
F7	P2	Clock-derived slots; free-text date field in meal-detail	Date picker + slot override control
F8	P2	Six camera controls stacked over the shutter	Reduce to mode + shutter; move the rest behind a gear sheet
F9	P2	Manual log path 5+ steps	Recents/favorites rail on Food tab; slim review for known items
F10	P2	Undo only in a 6 s toast	Durable undo on the logged card (swipe/long-press)

Table 8-2: Scan and logging flow (10 issues).

ID	Sev	Issue (short)	Direction
QA1	P1 (candi date)	Onboarding Back at step 1 exits without writing the completed key - re-onboarding loop risk	Device-verify; write the key on any exit path
QA2	P2	Same food re-logged from mounted search results	Reset search state on save
QA3	P2	Barcode route race: navigate before startBarcodeScan via module-global epoch	Device-verify; pass scan params explicitly
QA4	info	AI journeys hard-depend on a reachable LLM endpoint	Honest offline state; queue proposals (informational row, not counted in the 25)

Table 8-3: Onboarding and cross-app. QA1-QA4 correspond to F-QA1..F-QA6 with duplicates merged into T10 and the F-items; QA4 is informational, keeping the tracked total at 25.

8.2 Positive control

One behavior is explicitly worth protecting through the coming redesigns: finishWorkout refuses to complete a session with zero completed sets and cannot double-finish (packages/training/src/repository.ts:244-248). In a round full of 'the engine is better than the UX', this is the rare place where the guard, the message and the flow all agree, and the redesign of the workout finish flow (Ch. 9.2 pattern 5) must preserve it. The broader fairness note also stands: state hygiene across screens - loading, empty, error and corrupt-data handling - is unusually thorough in both evaluators' reads; the gaps concentrate in flow architecture, not in missing states.

9. Reproduction Roadmap

This chapter converts the research and the register into an implementable plan. Patterns are ordered so that the highest-leverage, lowest-risk land first, and each carries the issue IDs it resolves. The plan deliberately reuses what already exists: the dead 'captured' scaffold, the progression engine's five kinds, the log-correction layers, and the live workout's set table.

9.1 From Cal AI: thirteen patterns for the food side

#	Pattern	What changes in Nut AI	Resolves
1	Commit scans optimistically with undo	Insert the meal row as 'captured' at shutter time (the schema default already exists), dismiss straight to the tabs, show a 'logged' toast with a 5 s undo; the row renders as 'analyzing' in Home/Food and upgrades through the pipeline	F1, F2 partial
2	One scan path	Delete the quick/advanced toggle and its second render tree; low confidence triggers one inline follow-up question instead of a mode	F2, F8
3	Review becomes an editor, not a gate	The existing review UI reopens from the logged card as a bottom sheet with per-item gram chips	F1, F9, F10
4	Design the wait	Staged status (detecting, estimating, calculating) on the pending card and camera screen; never a bare spinner; cap and communicate the 180 s worst case	F1 UX
5	Multi-item result card	One card with per-item quantity chips - highest value for Indian mixed plates	F9
6	Camera-first home	Center shutter hero on Home; snap-to-log in ~3 taps; recents/favorites chips under the shutter	F9
7	AI chat as the entry editor	Fix the twelve failure modes: include past days in context, never drop tool calls silently, resolve names fuzzily with row previews instead of raw ids, implement qualitative sizes, surface per-op results, persist proposals in history, stream and schema-constrain the correction call, add a fallback chain	F4, F5, F6, A-items

#	Pattern	What changes in Nut AI	Resolves
8	Three equal verbs	Snap + Scan barcode + Describe meal presented equally on Home; describe covers dal/curd/mixed sabzi that photograph poorly	F9, J5
9	End-of-day correction brief	A gentle evening prompt: '3 meals logged - fix misses before the day closes'	F3 discovery, retention
10	Ring + streak dashboard and widgets	Remaining-calorie ring, macro bands, streak; expose the existing Android widgets around the ring	Retention
11	Honest estimates	Badge AI outputs as estimates; make per-100g macro overrides editable in meal-detail (totals already recompute from grams)	F3
12	One-tap re-log	Favorites/recent as one-tap chips (favorites-only already shipped in v0.3.0)	F9
13	Screenshot-friendly polish	Dark-option dashboard with one accent and a huge remaining numeral	Growth

Table 9-1: Cal AI-derived patterns.

9.2 From the workout leaders: fifteen patterns for the training side

#	Pattern	What changes in Nut AI	Resolves
1	Normalize the model	Routine = reusable template (exercises, sets, targets, rest); Program = start date + ordered days referencing routines; live session = routine instance + logged sets; relational rows with FK protection replace the twin JSON blobs and the cycle-day second language	T4, T6, T8, T11
2	Previous values inline	Ghost the prior session's weight/reps into every live input (the set table already has previous columns - widen history beyond same-routine)	T1 UX
3	Three-tap loop	Big steppers, one-tap set check, duplicate-last-set; keep the existing rest chip and auto-advance	T2 adjacent
4	Multi-axis progression	Per-exercise rule: add weight (+2.5 kg when all sets hit target), add reps (double progression), add set, or RPE-triggered - apply to bodyweight and timed lifts too	T1
5	'Next time' banner	After each exercise, show the computed next prescription with its reason, editable in place; also show it at routine launch instead of silent prefill	T1, T7, T8
6	Plain-language effort capture	Post-set prompt: Easy / Could do 2 more / Barely finished, mapped to internal RPE; advanced toggle exposes numeric RPE/RIR with a paired converter	T5
7	Point-of-use explainers	Every jargon chip (RIR, RPE, AMRAP, myo, cluster, tempo, 1RM) gets an expandable one-line definition where it appears - never a separate glossary screen	T5

#	Pattern	What changes in Nut AI	Resolves
8	Exercise picker, rebuilt	In-editor sheet with recents + favorites rail above search, muscle/equipment filters, and history-aware defaults replacing 20 kg x 10	T2
9	Builder reuses session cards	The routine builder renders the same set cards as the live session, including supersets and per-exercise notes	T2, T11
10	Plate calculator	Plates-per-side visual exists - surface it in the editor and session consistently	T2 adjacent
11	Auto rest timer	Per-exercise durations with presets; vibrate at zero (the rest chip exists - promote it)	Polish
12	Warm-up ramp generator	Empty bar to 40%x5 to 70%x3 to 90%x1, excluded from volume stats; first-class set types already exist	Advanced depth
13	Unit-aware planning	Editor fields convert and label per the user's unit setting	T3
14	Program as plan	start_date picker, generated calendar preview, 'Week N of M' indicator, and scheduled-vs-done adherence on the Progress tab	T6, T12
15	Motivation layer later	PR detection, weekly volume charts, streaks, year-in-review - after the model lands	T12, retention

Table 9-2: Workout-leader patterns.

9.3 Suggested phasing

Phase	Theme	Contents	Why this order
1	Scan flow rewrite	Patterns 9.1 #1-#4 + #11 (optimistic log, kill modes, editor-sheet review, designed wait, macro overrides)	Biggest visible gap vs Cal AI; reuses the dead scaffold; deletes the most code
2	Training model + editor	Patterns 9.2 #1, #8, #9, #13 (model normalization, picker, builder cards, units)	Unblocks everything else in training; fixes two P0s and the corruption risk
3	Progression + education	Patterns 9.2 #2, #4, #5, #6, #7 + 9.1 #7 (AI-edit repair)	Now that the model is relational, suggestions and multi-axis rules land cleanly
4	Retention surfaces	Patterns 9.1 #9, #10, #12, #13 + 9.2 #14, #15 (program plan, adherence, ring/streaks, motivation)	Compounds the earlier phases; safe to schedule last

Table 9-3: Phasing. Phases 1-2 resolve all five P0s.

9.4 Anti-patterns to avoid while rebuilding

- **Do not paywall the core loop.** Review synthesis punishes apps that gate logging itself; Cal AI's wall sits on AI scans while search/barcode stay free, and even that draws resentment.
- **Do not copy Fitbod's opacity.** Fully opaque auto-programming is the most common complaint about the AI-planner tier; every suggestion we make must show its reason and accept an override.
- **Do not let the search path rot.** Cal AI's 19-action search is survivable only because the camera carries the load; ours must stay under Cal AI's number, not match it.
- **Do not ignore user-provided grams.** It is a named Cal AI complaint; explicit user input must always win over the model's estimate.
- **Do not overclaim accuracy.** Badge estimates as estimates; the retention mechanism is cheap correction, not perfect vision.
- **Do not regress the positive control.** Keep finishWorkout's refusal of empty completion through any finish-flow redesign.

10. Prioritized Recommendations

If only ten things happen next, they should be these - ordered by user-visible value per unit of risk, each traceable to the register. Sizes are S (about a day), M (a few days), L (a week or more) for the existing codebase. This list is a decision surface for the owner, not a commitment; nothing below has been started.

#	Action	Resolves	Size
1	Optimistic scan logging: insert 'captured' at shutter, dismiss-on-capture, pending card in the day timeline, review reopened as an editor from the card	F1, F2 (core)	M-L
2	Delete the quick/advanced subsystem: one scan path, inline follow-up questions	F2, F8, big code removal	M
3	Macro overrides for logged items in meal-detail (per-100g fields; totals recompute)	F3	S
4	Stream + schema-constrain the AI correction call; remove the 30 s non-streaming timeout trap; surface per-op results and fuzzy name resolution with row previews	F4, F5	M
5	In-editor exercise picker sheet with recents/favorites and history-aware defaults; drag reorder	T2	M
6	Unit-aware routine editor fields	T3	S
7	Visible progression: next-time banner with reason at launch and post-exercise; extend progression to non-loaded lifts; widen history beyond same-routine	T1, T7	M
8	Plain-language effort capture + point-of-use explainers for RIR/RPE/set kinds/1RM	T5	S-M
9	Routine-delete cascade cleanup (or honest block) fixing program corruption	T4	S

#	Action	Resolves	Size
10	Program as plan: date picker, calendar preview, 'Week N of M', adherence row on Progress	T6, T12	M

Table 10-1: Ranked next actions. Items 1-2 alone put the scan flow at Cal AI parity on structure; 5-8 close the training complaints.

10.1 Decision points for the owner

- **Model normalization timing.** The relational routine/program model (9.2 #1) is the enabling dependency for the training fixes; doing item 7 before it would harden the JSON-blob design. Recommend accepting a larger Phase-2 diff once, rather than two rewrites.
- **AI provider requirements.** The edit-repair work assumes a provider that honors streaming and structured output; if the owner wants weakest-model compatibility, the fuzzy-name resolver and repair layer become mandatory rather than optional (they also fix the raw-id display).
- **Device QA path.** The emulator softemu wall is environmental. Options: a KVM-capable CI runner job for scripted smoke QA, a physical device run by the owner following the Chapter 7 journey scripts, or accepting the static-QA evidence class for one more wave.
- **Barcode corpus honesty.** The 116 curated barcodes remain clearly documented as curated-representative; if scan coverage complaints arrive, the OpenFoodFacts fallback already covers the long tail and can be promoted in UI copy rather than by inflating the local corpus.

This report intentionally changes nothing in the repository except itself. The register in Chapter 8 is the round's output; the roadmap in Chapter 9 is the round's opinion; the next round can begin from either without re-deriving any of it.